

Résumé des Travaux Pratiques : Architecture Logicielle

Haytham KENNOUZ

19 mars 2026

Table des matières

1	Introduction	2
2	TP 1 : Application E-commerce Monolithique	2
2.1	Objectifs	2
2.2	Réalisation Technique	2
2.3	Conclusion	2
3	TP 2 : Monolithe Modulaire	2
3.1	Objectifs	2
3.2	Concepts Clés	2
3.3	Conclusion	3
4	TP 3 : Architecture Distribuée	3
4.1	Objectifs	3
4.2	Composants de l'Infrastructure	3
4.3	Conclusion	3
5	Conclusion Générale	3

1 Introduction

Ce document présente une synthèse des trois travaux pratiques réalisés dans le cadre du module d'Architecture Logicielle. L'objectif global était de suivre l'évolution d'une application de la structure monolithique simple vers une architecture distribuée et scalable.

2 TP 1 : Application E-commerce Monolithique

2.1 Objectifs

Le premier TP portait sur la conception et le développement d'une application e-commerce de base suivant un modèle **monolithique traditionnel**.

2.2 Réalisation Technique

- **Stack** : Java 17, Spring Boot, Spring Data JPA, et PostgreSQL.
- **Modélisation** : Mise en œuvre d'entités liées (**Product** et **Category**) avec des relations *Many-to-One*.
- **API REST** : Création de points d'entrée pour les opérations CRUD (Create, Read, Update, Delete).

2.3 Conclusion

Ce TP a permis de maîtriser les fondamentaux de Spring Boot et de la persistance des données dans un environnement monolithique.

3 TP 2 : Monolithe Modulaire

3.1 Objectifs

Le second TP visait à transformer l'application monolithique en un **monolithe modulaire** pour améliorer la maintenabilité et réduire le couplage.

3.2 Concepts Clés

- **Modularité** : Organisation du code par domaines métier (*package-by-feature*) : Customer, Product, Order, Category.
- **DTO et Mapping** : Utilisation de **MapStruct** pour séparer les modèles de données internes des modèles exposés via l'API.
- **Communication Inter-modules** : Injection de dépendances entre services pour valider les commandes.

3.3 Conclusion

L'architecture modulaire offre un bon compromis entre la simplicité de déploiement d'un monolithe et la clarté structurelle des microservices.

4 TP 3 : Architecture Distribuée

4.1 Objectifs

Le dernier TP portait sur le passage à une **architecture distribuée** utilisant des conteneurs, le load balancing et la communication asynchrone.

4.2 Composants de l'Infrastructure

- **Docker** : Orchestration de 7 services (App1, App2, MongoDB, Redis, RabbitMQ, Nginx).
- **Nginx** : Configuré comme Load Balancer avec l'algorithme *Round-Robin*.
- **Redis** : Mise en cache des résultats de recherche pour améliorer les performances.
- **RabbitMQ** : Gestion des flux de données asynchrones (notifications de commandes) pour découpler les services.

4.3 Conclusion

Cette étape a démontré comment gérer la scalabilité et la résilience d'un système complexe grâce aux technologies de conteneurisation et aux bus de messages.

5 Conclusion Générale

Le parcours à travers ces trois TP a permis d'appréhender concrètement les défis de l'architecture logicielle moderne. De la simplicité du monolithe à la robustesse des systèmes distribués, chaque étape a apporté des solutions spécifiques aux problèmes de scalabilité, de performance et de maintenance.